① Used to find element or pairs of elements from Array

HASH MAP

** Revise VS Code notes from github {Hashing}

Leet-1 Two Sum

- Can't use 2 ptrs as array not sorted
- Refer Leet
- No subsequence or substring required so sliding or prefix not used.

T- O(n)
S- O(n)

3-Sum & 4-Sum → * (Nested loops) + HASH SET

* Sorting + End ptrs (optimal)

Leet-41. First Missing Positive → HashMap

T- O(n)
S- O(n)

• Optimal → Cyclic Sort (Index Correction)

T- O(n)
S- O(1)

Leet-2367 Number of Arithmetic Triplets

HASH-MAP → T- O(log(n)) S- O(n)

```
unordered_map<int, int> m;
for (i=0 to n)
    m[nums[i]]++;

for (i=0 to n)                          // given in Qs
    x = nums[i] + diff;
    y = nums[i] + (2 * diff);
    if (m.find(x) != m.end() && m.find(y) != m.end())   // instant finding in HashMap
        Count++
```

HASH-SET → T- O(log(n)) S- O(n) → No need to track values so better use Hash Set.

```
unordered_set<int> S(nums.begin(), nums.end());
for (int num : nums)
    if (S.count(num+diff) && S.count(num + 2*diff))
        Count++
```

* In 2 Sums we had to return indices also tracked using HashMap (key, values) but Here in above Qs we just need to return Count so using Hash Set as no need to track indices

Ajanta PRODUCTS Officer

Leet-169 Majority Element

Refer Leet

* Sorting → Brute force — T- O(n log n), S- O(log n)
* Hash map → T- O(n), S- O(n)
* ★ optimal * Boyer-Moore voting Algo → T- O(n), O(1)

```
int e; → Current Candidate
int c=0; → Candidate's vote Count
for(i=0 to n)
    if(c==0)          } New Candidate arrived
        e = nums[i]
        c=1
    else if(e==nums[i])   } Count votes of pre existing candidate
        c++;
    else
        c--;          → if another number appears, reduce vote of pre existing Candidate, if pre existing Candidate's vote get reduced to '0', new Candidate will be chosen
```

Dry Run: 2, 2, 1, 1, 1, 2, 2

- i=0 e=2 (current Cand) c=1 i/p=2
- i=1 e=2 c=2 i/p=2
- i=3 e=2 c=1 i/p=1 another no arrived so reduce vote
- i=4 e=2 c=0 i/p=1
- i=5 e=1 c=1 i/p=1

Leet-229 Majority Element-II

Refer Leet

* HASH MAP → T- O(n), S- O(n) → Returns vector as more than 1 element Can be any
* Boyer-Moore → T- O(2n), S- O(1)

Max 2 elements Can only be having majority of more than n/3 ($\frac{n}{3}+\frac{n}{3}+\frac{n}{3}=n$) → so more than 2 elements Can't have ($>n/3$)

```
int K1 = INT_MIN;
int K2 = INT_MIN;
int C1 = 0
int C2 = 0
int n = num.size();
// Find Candidates using Boyer-Moore Voting Algo
for(i=0 to n)
  if(C1=0 && n[i] != K2)   → C1=0 means Current element is ^New Candidate
      K1 = n[i]            → n[i]=K2 means current element is not K2 (2nd Candidate)
      C1 = 1               → Assign Current element as Candidate 1
  else if(C2=0 && n[i] != K1)
      K2 = n[i]
      C2 = 1
  else if(n[i] = K1)       → 1st Candidate arrived so increase vote
      C1++
  else if(n[i] = K2) C2++
  else C1--; C2--;         → Current element is neither 1st nor 2nd Candidate so reduce both's vote
```

// Now check whether Candidates are actually in majority or not may be element is more in number than other element but not more than n/3

```
int c1 = 0
int c2 = 0
for (i:0 to n)
    if (n[i] = k1)
        c1++
    if (n[i] = k2)
        c2++
```

continued from here

```
if (c1 > n/3 && c2 > n/3)
        return {k1, k2}
if (c2 > n/3)
        return {k2}
if (c1 > n/3)
        return {k1}
return {}
```

Leet-347 Top K Elements

Better

Hashmap + vector

T- O(n log n)

- Creating mp → O(n)
  - insertion takes O(1) for each element & n no. of elements
- Copy mp into vector → O(u)
  - no. of unique elements
- Sort vector → O(u log n) ★★★ Max
- Push_back into ans vector = O(k)

S- O(n)

- map = O(u)
- sorting vector = O(u)
- Ans vector = O(k)

worst Case → u = n

So, O(u) + O(u) + O(k)

= O(u)

= O(n)

Starts from here ↓

```
unordered_map<int, int> m;
for(int n : nums)
    m[n]++
```

→ no use of ordered mp as they stores elements sorted ascending to keys not values of mp

```
vector<pair<int,int>> v(m.begin(), m.end())
```

→ Transfer whole mp into vector of pairs for sorting accordy to values of mp

```
sort(v.begin(), v.end(), [](auto &a, auto &b){
    return a.second > b.second
});
```

lambda comparator

→ Sorted accordy to descending order

a.second < b.second then it would be ascending

Sort(v.begin(), v.end(), greater<int>())

↑ this can't be used as it's pair

v.Sort Highlight this

```
vector<int> ans;
for (int i=0; i<k)
    ans.push_back(v[i].first);
```

Optimal

T- O(n)

- Hash map → O(n)
- fill Bucket → O(n)
- Traverse Bucket → O(n)

S- O(n)

- map - O(n)
- bucket - O(n)

★ Bucket

Step 1 : Create map for frequencies

Step 2 : Create Bucket of size → n+1 (size of array given in question)

```
vector<vector<int>> bucket(nums.size()+1)
```

nums = {5,5,5,5,5,5}

freq of 5 = 6

So Bucket

{0} : 0
{1} : 0
{2} : 0
{3} :
{4} :
{5} :
{6} : 5

(freq, element)

Step 3 : (fill elements in bucket)

```
for (auto &p : mp)
    bucket[p.second].push_back(p.first);
```

** Highlight this

p.second → freq of element

p.first → element

Ex → [1 1 2 1 3 2]
↳ size = 6

map → m[1] = 3
m[2] = 2
m[3] = 1

Step 4 ~~Traverse Backward~~

Uses vector of vector not vector of pairs to create bucket bcs :—

✓ `vector<vector<int>> bucket(nums.size()+1)`

Bucket size = 7 buckets

bucket[0] = {}
bucket[1] = {3}
bucket[2] = {2}
bucket[3] = {1}
bucket[4] = {}
bucket[5] = {}
bucket[6] = {}

(already aligned according to frequencies)

(element, freq)

bucket[0] → {}
bucket[1] → {4, 7, 10} → multiple elements having freq as 1
bucket[2] → {5, 8}
bucket[3] → {1}
bucket[4] → {9, 2}

xx `vector<pair<int,int>> bucket(nums.size()+1)`

Suppose element '1' has freq 2
bucket[2] = {1, 2}
Again if 4 also come 2 times
bucket[2] = {4, 2}

replaced → Can't work if multiple elements get same freq.

Step 4 Back traversal

```
for(int = num.size() to 1 ; i--)
    for (int n : bucket[i])
        ans.push_back[num]
        if (ans.size() == k)
            return ans;
```

Bucket size → nums.size() + 1
so traverse from num.size()

bucket[i] = element
↳ n

we use this traversal not like

```
for(int i= num.size to num.size()-k)
```

as let say bucket size = 7

But Bucket[6] = {}
Bucket[5] = {}
Bucket[4] = {}
bucket[3] = {1}
bucket[2] = {2}
bucket[1] = {3}

(It will fetch these 2 only for k=2)

Better
Min Heap
Priority Queue → $T = O(n \log k)$

$S = O(n)$

→ Check leet.

HASH-MAP

#Leet-451 Sort char by freq.

i) Priority Queue → T: O(n log K)
- n → total no of chars
- K → no of unique chars

Each Push() in Heap (Priority Queue) costs O(log K)

Check Solutions (codes) on Leet

T: O(n log n)

S: O(n)

ii) HashMap Sorting using pair vector → T: O(n log n), S: O(n)

optimal iii) Bucket Sort → T: O(n), S: O(n)

Task Scheduler | Rising Brain | GfG

1st Approach Priority Queue →

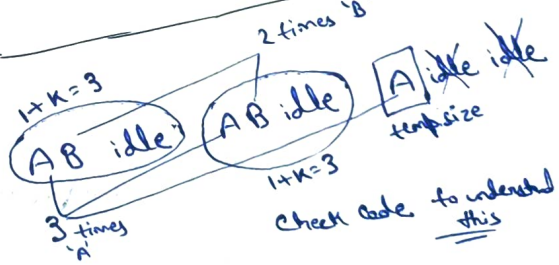
T: O(N)* → Size of queue is fixed (26)

or O(N log M) → Push-Pop operations in queue

S: O(1)

Tasks = [A, A, A, B, B]

K = 2



Check code to understand this

```
int leastInterval(int k, vector<char> &tasks){
    int count = 0                          → find Answer
    vector<int> freq(26, 0)                → only Caps Alphabets, so instead of map will create Hash Array / Hash vector of fixed size 26, initializing '0' frequency for each letter.
    for(auto task : tasks){
        freq[task - 'A']++;                → Count frequencies of tasks available 'A' & 'B'
    }                                      (Caps Alpha)
```

Ajanta PRODUCTS Officer

```
Priority_queue<int> pq;
```
→ max heap

```
for(auto t : freq) {
    if(t>0){
        pq.push(t);
    }
}
```

Putting frequencies into queue

note → Hash array have 26 frequencies

index:	A	B	2	3	4	----	26
freq:	3	2	0	0	0	----	0

↳ this 2 only will go into queue

```
while(!pq.empty()){
```

A B Idle → 1st cycle | A B Idle → 2nd cycle | A → 3rd cycle

As long as tasks present in queue i.e queue not empty, go for next cycle.

```
vector<int> temp;
```
→ for each cycle we will store modified freq into temp then transfer into queue, can't update queue in middle of cycle

```
for(int i=0; i<k+1; i++){
```
if k=2 then i=0,1,2

k+1 = 2+1 = 3 times

so in each cycle 3 positions → A, B, Idle

```
if(!pq.empty()){
```
if Heap has a task then start execution

A=3 → will execute it here

```
int curr = pq.top();
```
→ most freq element chosen A='3'

```
pq.pop();
```
→ Pop it out of queue so, in current cycle A got executed now it won't come again in this same cycle so pop out

```
curr--;
```
→ A=2 (modified)

```
temp.push_back(curr);
```
→ Put modified freq into temp

Similarly for both loops A, B modified freq will be stored into temp then later after completion of cycle it will be transfered to queue

```
    }
}
```

```
for(auto f : temp)
    if(f != 0)
        pq.push(f)
```
→ Check modified freq stored in temp if it's not '0' then push back into queue → {A,B} → {2,1}

```
}
if(pq.empty())
    count += temp.size();
else
    count += k+1;
```
→ if Heap comes to end → AB Idle | AB Idle | (A) ← last cycle, ↳ temp.size

→ (A B Idle, k+1=3) | (A B Idle, k+1=3) | A ↳ temp.size

2nd Approach → Greedy

T: O(n) → every task being visited once

S: O(1)

Time → O(n) for creating Hash array

O(26 log 26) for sorting

O(26) = O(1) for loop

```
if(task.size() == 0)
    return 0
```
→ Edge Case: no tasks

```
vector<int> freq(26,0)
```
→ Hash vector

```
for(auto task : tasks)
    freq[task - 'A']++
```

```
Sort(freq.begin(), freq.end(), greater<int>());
```
→ Sort in descendig.

```
int maxfreq = freq[0];
int gaddha = maxfreq - 1;
int idle = gaddha * k;
```
→ Sorted in des so 1st element will have max freq.

k=2

A:4 C:1

B:4 D:1

A _ _ A _ _ A _ _ A

gaddha = 4 - 1 = 3

total spots = gaddha * k

= 3 × 2 = 6

```
for(int i=1 to 25)
    idle -= min(freq[i], gaddha);
```

let's fill up 'B'

A B _ A B _ A B _ A

3 B's filled

1 is left

So remaining spots = 6 - 3 = 3

no of gaddha

fill 'C'

A B C A B _ A B _ A

remaining spots = 3 - 1

try for 'C'

So, spots = spot - smaller of freq & gaddha

⇒ spot -= min(freq[i], gaddha)

```
if(idle > 0)
    return tasks.size() + idle;
else
    return tasks.size();
```

① A:4 C:1 B:4 D:1 } 10

A B C A B D A B _ A (B)

1 B left to fill

only 3 B's are filled

idle → total = task.size() + idle

= 10 + 1 = 11 ✓

② A:4 C:2 B:3 D:1 } 10

A B C A B C A B D A

total = tasks.size() = 10 ✓

③ A B C A B D A B _ A

A:4 C:1 B:3 D:1 } 9

total = 9 + 1 = 10



Leet - 1838 Freq of Most freq Element

↳ * Binary Search
* sliding window problem.